RPN–CG 的 C++ 实现说明与测试记录

ROPTLIB 项目

2026 年 10 月 14 日

摘要

本文依据 Solvers/RPNCG.cpp、Solvers/RPNCG.h 及相关问题类和测试入口，记录从 Matlab 思路移植到 ROPTLIB 的 RPN–CG 实现：类关系、接口和状态、外层与两个内层求解过程、临时对象的生命周期，以及现有测试能够证明的结果。未取得的跨算法和跨语言数值不以估计值代替。

1 问题、类关系及接口

测试对象是 Stiefel 流形上的稀疏 PCA：给定 $B \in \mathbb{R}^{m\times n}$，求

$$\min_{X\in\mathbb{R}^{n\times p},\ X^{\mathsf{T}}X=I_p} F(X) = f(X) + g(X) = -\|BX\|_F^2 + \lambda\|X\|_1. \tag{1}$$

StieSPCA 继承 Problem，提供 f, g、欧氏梯度 $-2B^{\mathsf{T}}BX$、欧氏 Hessian 作用 $Y \mapsto -2B^{\mathsf{T}}BY$、软阈值近端映射 ProxW 和其广义导数作用 CalJW。$L = 2\|B\|_2^2$ 在问题构造时通过奇异值分解得到。Stiefel 负责流形随机初值和回缩；测试通过 ChooseParamsSet4() 选择 polar 型回缩。

关系	职责
Solvers → SolversNSM → RPNCG	通用计时/计数 → 非光滑求解器接口 → 本算法
RPNCG → Problem (StieSPCA)	借用目标、梯度、Hessian 作用和近端算子
Problem → Manifold (Stiefel)	借用约束流形和回缩

此处箭头对前三类表示继承，对后两行表示使用/关联，并非所有权转移。RPNCG(prob,initialx) 调用父类初始化，设置默认参数并复制初始迭代点；析构函数是默认实现。

1.1 成员函数与参数

函数	实际作用
SetDefaultParams()	调用 SolversNSM::SetDefaultParams() 后设定算法参数。
Run()	外层迭代、活动集划分、步长调整、回缩和至多三次回溯，记录部分统计。
FindDir()	带正则的半光滑 Newton–CG 解乘子方程,返回近端方向 v 并更新 Dinitial。
ApplyBB()	利用 Prob->EucHessianEta 计算二阶算子作用。
tCG()	在活动坐标上作投影 CG 修正 w，返回状态码。
UpdateData(), PrintInfo(), PrintFinalInfo()	前者当前为空；后两者直接调用非光滑求解器父类的打印函数。

文件内辅助函数 ProxAt、JacobianAt 将步长 t 转换为 ProxW/CalJW 所需的标量权重 $1/t$；Aop/AstarOp 处理约束线性化，GatherVector 按索引取子向量，BuildBbar 构造约束投影矩阵，cmidx 处理列主序索引。这些是文件作用域函数，不是类成员。

成员（默认值）	用途
$\tau = 100$, $\gamma = 0.01$, $\vartheta = 0.01$	tCG 中非活动坐标的二次惩罚、能量/曲率判断。
$\rho_1 = 10^{-3}$, $\rho_2 = 0.5$	回溯充分下降阈值和步长缩减比。
$w_1 = 1.1$, $w_2 = 0.9$	外层近端步长 t 的放大/缩小系数。
$\theta = 0.5$, $\kappa = 0.1$	tCG 残差的超线性/线性停止阈值。
max_innit= 200	tCG 最大迭代数；每步按活动集大小重新设置。
max_inner_iter	在头文件声明，但当前实现不赋值、不使用。
继承的 Tolerance, Max_Iteration, Verbose 等	停止参数和输出配置；x_1, x_2, f_1, f_2、iter, ComTime, nf, ng, nR, nH 等由基类存放。

头文件还有独立的 RPNCGParams 结构体（其中不含 τ）；当前 Run() 读取的是 RPNCG 自身的成员，没有使用该结构体。RPNCGTCGStatus 中 -1 表示达到内迭代上限，$0,1,2,3$ 分别是两种初期退出、负曲率和能量判据退出，$4,5$ 分别对应线性和超线性残差判据。

2 运行逻辑

2.1 外层迭代

Run() 先调用父类 Run() 初始化计时/输出缓存，计算 $F(x_1)$ 和欧氏梯度，设 $t = 1/L$（仅当问题可转换为 StieSPCA；否则取 1），初始化零乘子矩阵 $D \in \mathbb{R}^{p\times p}$。在 $\mathrm{err} >$ Tolerance 且 iter $<$ Max_Iteration 时执行：

1. 根据前一轮 $\|v\|_F$ 和 tCG 状态收紧 innertol；调用 FindDir 求 v，以 $\|v\|_F$ 记作 n_v。乘子 D 跨外层迭代热启动。
2. 逐个列主序位置 k 判断：若 $(x+v)_k \neq 0$ 且 $|x_k| \geq n_v$，放入 id1，否则放入 id2；将 tCG 上限设置为 $\max(1, \min(200, \mathrm{round}(1.2|\mathtt{id1}|)))$。
3. 计算 $B_\lambda = xD$、tmp11$= x^{\mathsf{T}}B_\lambda$，调用 tCG 求仅定义在 id1 上的修正 w；拼出 $u_{\mathtt{id1}} = v_{\mathtt{id1}} + w$、$u_{\mathtt{id2}} = v_{\mathtt{id2}}$，令 $n_u = \|u\|_F$。
4. 若 $(4 + 1/t)n_u < n_v$ 或状态为 0，令 $t = \max(t_0, w_2 t)$；否则状态 ≤ 4 时令 $t = w_1 t$。状态 ≥ 5 使用带 flag 的特殊分支（可能回到上一轮保存的 $x_{\mathrm{old}}, u_{\mathrm{old}}$），普通分支用 $x_2 = R_{x_1}(\alpha u)$，α 从 1 开始按 ρ_2 缩减，至多追加三次回溯。
5. 计算新梯度，设 $\mathrm{err} = \min(n_v, n_u)$，更新 x_1, f_1 和梯度，累计 iter、目标/梯度/回缩调用次数；遇到新目标值 NaN/Inf 则中止。循环结束记录 ComTime。

这里的停止条件是 Run() 自行实现的绝对方向范数判据,不调用父类 IsStopped();因此基类 Stop_Criterion、TimeBound 和 StopPtr 不控制此循环。

2.2 乘子半光滑 Newton–CG：FindDir

令 $\mathrm{sym}(Z) = (Z + Z^{\mathsf{T}})/2$，源码定义

$$A_x(\eta) = -\tfrac{1}{2}(\eta^{\mathsf{T}}x + x^{\mathsf{T}}\eta), \qquad A_x^*(\Lambda) = -x\,\mathrm{sym}(\Lambda), \tag{2}$$

$$B(D) = x - t\big(\nabla f(x) - A_x^*(D)\big), \qquad F(D) = A_x\big(\mathrm{prox}_{tg}(B(D)) - x\big). \tag{3}$$

以传入的 D 为初值，使用 CalJW 作用形成 Jacobian–向量积；每个 Newton 步用最多 30 次 CG 求正则化线性方程（正则系数随残差变化），最多 1000 次 Newton 更新。接受条件使用长度为 5 的非单调残差窗口；拒绝时依据 ρ 调整正则系数，并尝试投影修正/残差步。循环判据是 $\|F(D)\|_F^2 >$ innertol，返回 $v = \mathrm{prox}_{tg}(B(D)) - x$、迭代次数及新乘子；这不是直接求 tCG 的修正量。

2.3 约束投影 tCG 与二阶算子

ApplyBB 对 Y 计算

$$\mathcal{B}_x(Y) = \nabla^2 f(x)[Y] + Y(x^{\mathsf{T}}B_\lambda) + x\,\mathrm{sym}(Y^{\mathsf{T}}B_\lambda). \tag{4}$$

BuildBbar 对称矩阵空间的 $p(p+1)/2$ 个标准正交基 S_j 构造 $\bar{B}_{kj} = (xS_j)_{\mathtt{id1}[k]}$；对角基系数为 1，非对角基系数为 $1/\sqrt{2}$。令 $D_B = \bar{B}^{\mathsf{T}}\bar{B}$，通过伪逆构造投影 $P(z) = z - \bar{B}D_B^{\dagger}\bar{B}^{\mathsf{T}}z$。tCG 先检查基于 $\mathcal{B}_x(v)$、$g(x+v) - g(x)$ 的能量条件；若 id1 为空立即返回空修正。其余情况下在投影子空间从 $w = 0$ 开始做 CG，检查负曲率、能量下降及 $\|r_j\| \leq \|r_0\|\min(\|r_0\|^{\theta}, \kappa)$；未满足而迭代耗尽则返回零修正。注意源码在循环尾更新 inner_it，某些提前退出的日志计数可能小于实际完成的 CG 步数，不能据此精确核算 Hessian 次数。

3 内存管理与统计边界

Prob、Mani 是基类保存的非拥有型指针：调用方须确保问题和流形比求解器活得更久；测试中三者均为栈对象。x1/x2、乘子 D、子向量和中间矩阵使用 Variable/Vector 值对象及局部 std::vector<integer> 管理，函数返回值和容器按 C++ 生命周期释放，不在 RPNCG 中手动 new/delete。BuildBbar 直接生成大小为 $|\mathtt{id1}| \times p(p+1)/2$ 的矩阵，避免显式存储 Matlab 风格的三维中间张量；另需保存 $p(p+1)/2$ 阶的

Gram 矩阵及其伪逆。GatherVector 会复制活动集数据，ApplyBB 及矩阵乘法也会产生临时 Vector，不能声称整体零拷贝。

基类的 nf、ng、nR 分别在 Run() 的目标值、梯度、回缩调用点显式自增；但 FindDir 内 ProxW/CalJW、tCG 内 g() 以及 ApplyBB 中 EucHessianEta 的调用未通过专用计数器完整统计。尤其 nH 没有在该实现中增加，不能把打印的零当作没有二阶计算。

4 测试记录与对比边界

4.1 现有 C++ 用例

入口 test/mainRPNCG.cpp 使用随机种子 2；test/testRPNCG.cpp 取 $(n, m, p) = (800, 50, 5)$、$\lambda = 0.8$，对高斯矩阵每列去均值并归一化，再随机生成 Stiefel 初值。设置 Max_Iteration $= 1000$、Tolerance $= 10^{-6}$、Verbose=ITERRESULT，运行一次 solver.Run()。对仓库已有 build/bin/testRPNCG 可执行文件的一次实际运行记录如下（程序逐步打印计时，因此仅作单次样例，非稳定基准）：

方法	外层步数	最后输出的 F	程序内用时 (s)	最后 n_v	最后 n_u
RPN–CG (已有 C++ 二进制)	147	-3.850512×10^{1}	0.558956	8.693090×10^{-8}	1.513399×10^{-6}

末次 $\min(n_v, n_u) < 10^{-6}$，故按源码循环判据退出；这是现有二进制的观察值，未验证其编译内容是否与当前源码完全一致。日志未给出 nf/ng/nR 的最终总数，也没有可靠的 Hessian/近端调用次数，故这些数值暂记为未记录，而非由迭代次数推断。

4.2 同口径比较所需补测

比较对象	时间 (s)	主要计算次数	效果/可比性
C++ RPN–CG（上述用例）	0.558956（单次）	外层 147；nf/ng/nR 未记录	$F \approx -38.50512$
C++ 一阶方法（如 IARPG/IRPG）	未取得	未取得	未在同一数据和初值下运行
Matlab 二阶 RPN–CG	未取得	未取得	仓库中未找到对应的 Matlab RPN–CG 驱动程序/结果

仓库 test/TestStieSPCA.cpp 中虽有 IARPG 测试，但默认 $(n, m, p) = (1000, 20, 5)$、$\lambda = 2$，与本用例不同，不能直接做效率比；Matlab/ForCpp/CTestStieSPCA.m 配置的是一阶方法（$n = 1000, p = 17$ 等），不是所要求的 Matlab 二阶基线。当前会话尝试启动 Matlab 获取版本信息时未在 8 秒内返回，因此没有可验证的 Matlab 二阶运行值。

为完成 req.txt 的两项对比，应固定同一个 B 和 X_0（将输入数据及初值在两端共享），统一 λ、回缩、精度、最大迭代数和停止判据，并在相同硬件/线程数下分别多次计时（报告中位数）。至少记录最终目标值、$\min(n_v, n_u)$ 或共同定义的残差、外层次数、总耗时、目标/梯度/回缩次数，以及 Hessian–向量积、近端映射、Newton 和 CG 的实际总调用次数。对 Matlab 需先取得相应二阶代码及可运行环境；对 C++ 需先在同一测试入口运行一阶求解器。若仅允许修改 TeX，这些测试入口和计数埋点不能在本次工作中补写；故不作无依据的速度优劣结论。